

Inside stsmell

How a Tonel file on disk becomes a code-smell finding on stdout – the eight stages, the data that moves between them, and the Smalltalk-specific decisions that shaped each one.

Package stsmell 0.1.0 **Language** Python ≥ 3.10 **Parser** tree-sitter-tonel-smalltalk **Modules** 21
Rules 18

Orientation

what the tool is, and the one law the codebase obeys

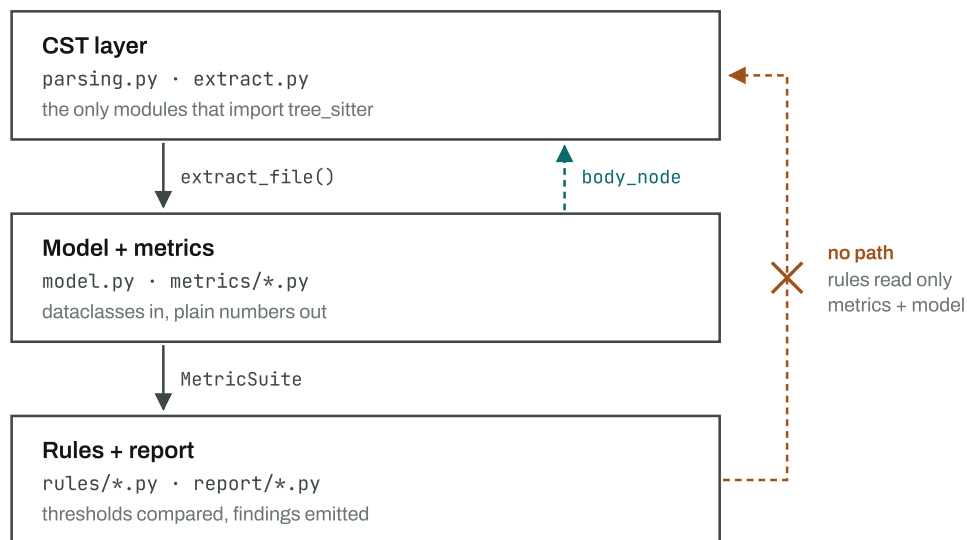
`stsmell` is a command-line code-smell detector for Pharo/Smalltalk. It reads **Tonel**-format source — the plain-file layout Pharo uses when a package is exported to Git — and reports findings as text, JSON or SARIF, exiting non-zero when it finds something.

That framing matters for the architecture. Pharo's own analysis tools, Moose and SmallLint, run *inside a live image*: they reflect over real class objects and real compiled methods. `stsmell` has no image. It has bytes on disk, a tree-sitter grammar, and whatever it can reconstruct from them. Every design decision downstream follows from that constraint — it must recover a usable model of a class hierarchy from a directory of text files, and it must be honest about what it could not recover.

The layering law

One rule governs the whole codebase, and it is worth internalising before reading any module: **rules never touch the concrete syntax tree**. Rules consume the extracted model plus precomputed numbers, nothing else. The docstring in `model.py` states it directly, and the module graph enforces it — only `parsing.py` and `extract.py` import `tree_sitter`.

The payoff is testability. A rule is a pure function from numbers to findings, so `tests/test_rules.py` can construct a `MethodMetrics` by hand and assert on the output without parsing anything. The cost is one deliberate exception, described below.



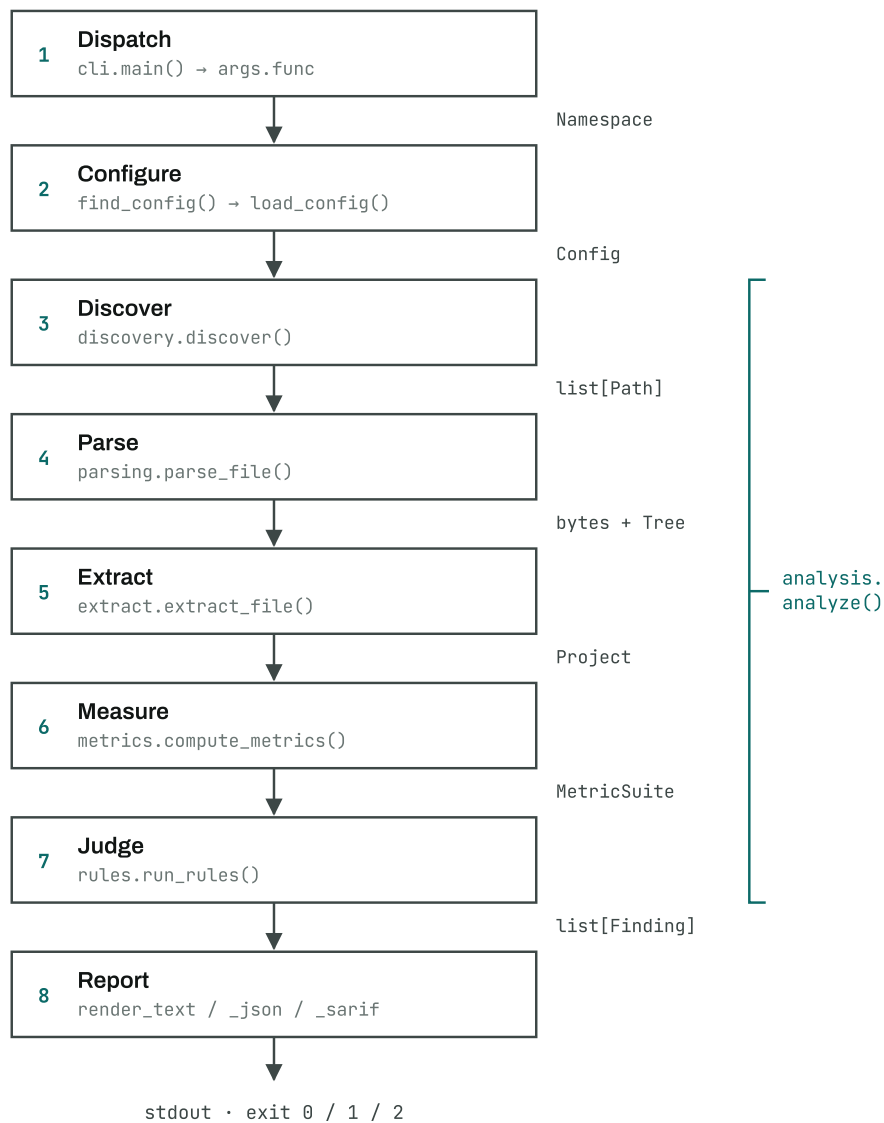
The containment boundary. Extraction translates the tree once, downward. Metric code is allowed a single reach back up — the `tree_sitter.Node` parked on `MethodDef.body_node` — because computing complexity requires walking the body. Rules get no such door.

Module map

DRIVER cli.py argparse, four subcommands, exit codes analysis.py the pipeline in fifteen lines config.py TOML discovery, thresholds, severity order	FRONT END discovery.py find Tonel files, apply excludes parsing.py tree-sitter setup, source normalisation extract.py CST → model translation model.py ClassDef, MethodDef, ParsedFile, Project	MEASUREMENT selectors.py the curated branching-selector sets metrics/method.py MLOC, CYCLO, NBLK, envy, chains metrics/klass.py WMC, TCC, CBO, accessor ratio metrics/hierarchy.py DIT, NOC, ancestors metrics/suite.py runs all three over a Project
JUDGEMENT rules/base.py Rule, Finding, the registry, run_rules rules/method_rules.py nine method-level smells rules/class_rules.py six class-level smells rules/smalltalk_rules.py three with no Java analogue	OUTPUT report/text.py ANSI-coloured terminal report report/json_report.py summary + findings report/sarif.py SARIF 2.1.0 for code scanning	TESTS test_extract.py CST → model translation test_metrics.py model → numbers test_rules.py numbers → findings test_cli.py end to end, exit codes fixtures/ one Tonel file per smell, plus clean/

The pipeline at a glance

Everything below is a walk down this spine. The numbered sections of this document *are* these stages, in execution order; unnumbered sections after them are reference material rather than steps.



The spine. Stages 1–2 belong to the CLI; stages 3–7 are the reusable core, wrapped by `analysis.analyze()` so any embedder gets the same pipeline without argparse; stage 8 turns the result into bytes and an exit status.

SRC/STSMELL/ANALYSIS.PY – THE WHOLE PIPELINE

```

def analyze(root: Path, config: Optional[Config] = None) → AnalysisResult:
    config = (config or Config()).with_defaults(registry())
    project, parse_errors, read_errors = build_project(root, config)
    suite = compute_metrics(project)
    findings = run_rules(suite, config)
    return AnalysisResult(...)
  
```

01 Dispatch

src/stsmell/cli.py

The entry point declared in `pyproject.toml` is `stsmell = "stsmell.cli:main"`. `main()` builds the parser, parses `argv`, and hands off to the function that the chosen subparser stashed on the namespace with `set_defaults(func=...)`. There is no dispatch table and no `if/elif` chain – the subparser *is* the dispatch.

Four subcommands exist. Three of them – `analyze`, `metrics`, `baseline` – take the same positional path plus `-c/--config` and repeatable `-e/--exclude`, shared through a local `add_common(sub)` helper. The fourth, `rules`, takes no path because it inspects the registry rather than any source.

<code>analyze</code>	Run the pipeline and report findings. Adds <code>-f/--format</code> (<code>text · json · sarif</code>), <code>--min-severity</code> and <code>--baseline</code> .
<code>metrics</code>	Run the same pipeline but dump the raw metric layer instead of findings. Adds <code>-f/--format</code> (<code>csv · json</code>) and <code>-s/--scope</code> (<code>method · class</code>). This is the calibration tool.
<code>rules</code>	List every registered rule with its severity, description and active thresholds. Reads no source.
<code>baseline</code>	Freeze today's findings to a JSON file so that later runs report only what is new. Adds <code>-o/--output</code> .

Exit codes and error handling

The three exit codes are module constants, and they are the contract CI depends on:

<code>EXIT_OK = 0</code>	No findings survived filtering – or the command was informational (<code>metrics</code> , <code>rules</code> , <code>baseline</code>), which always succeed.
<code>EXIT_FINDINGS = 1</code>	At least one finding at or above <code>min_severity</code> remains. This is what fails a pull request.
<code>EXIT_ERROR = 2</code>	A bad config or a missing path. Distinct from 1 so a pipeline can tell "smells found" from "the tool broke".

`main()` wraps the dispatch in a small `try` that converts `ConfigError` and `FileNotFoundError` into a message on `stderr` plus exit 2. It also catches `BrokenPipeError` and returns 0 – that is the `stsmell analyze src/ | head` case, where the downstream process closing the pipe is normal, not a failure.

NOTE

Per-file read failures are *not* errors. `build_project()` catches `OSError` and `UnicodeDecodeError` per path, collects them into `result.read_errors`, and keeps going. `cmd_analyze` prints them to stderr after the report so they never corrupt machine-readable stdout.

02 Configure

`src/stsmell/config.py · cli._load()`

Configuration is resolved before any file is opened, and it is assembled from three sources with a strict precedence order.

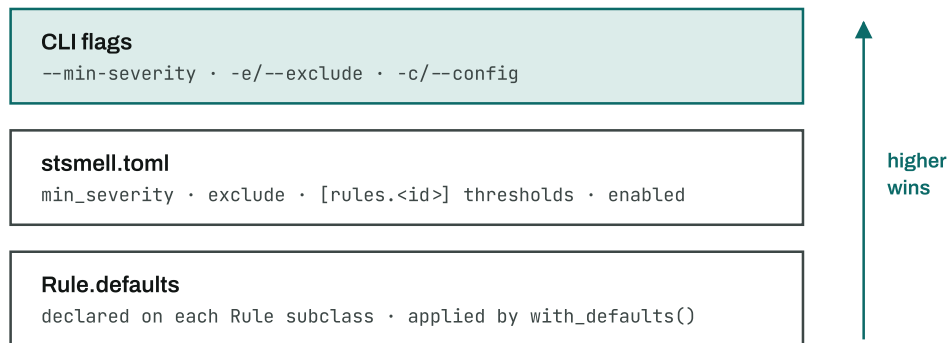
Discovery comes first. Unless `-c` names a file explicitly, `find_config()` walks *upward from the analysed path* – not from the current working directory – looking for `stsmell.toml` or `.stsmell.toml`. Analysing `src/Kernel/` inside a checkout therefore picks up the config at the repository root, exactly as a developer expects.

`SRC/STSMELL/CONFIG.PY`

```
def find_config(start: Path) → Optional[Path]:
    start = start.resolve()
    candidates = [start] if start.is_dir() else [start.parent]
    candidates.extend(candidates[0].parents)
    for directory in candidates:
        for name in CONFIG_FILENAMES:          # stsmell.toml, .stsmell.toml
            candidate = directory / name
            if candidate.is_file():
                return candidate
    return None
```

`load_config()` then parses it with `tomllib` (with a `tomli` fallback for Python 3.10), validating as it goes: an unknown `min_severity`, a `[rules]` key that is not a table, or malformed TOML each raise `ConfigError` and become exit code 2. Back in `cli._load()`, command-line flags are layered on top – `--exclude` patterns are *appended* to whatever the file listed, while `--min-severity` *replaces* it.

The last layer is applied inside `analyze()`, not the CLI: `Config.with_defaults(registry())` walks every registered rule and fills in that rule's declared `defaults` dict for any threshold the user did not set, plus `enabled = True`. This is why a rule's defaults live on the rule class rather than in a central table – a new rule ships its own thresholds and needs no edit anywhere else.



Three layers, one direction. Note the asymmetry: the TOML layer is the only one that can change a rule threshold or disable a rule. There is deliberately no `--max-lines` flag — thresholds are project policy and belong in a file under version control.

GOTCHA

`stsmell rules` does not run `find_config()`. It takes no path argument, so with no `-c` it constructs an empty `Config()` and reports the built-in defaults — even when a `stsmell.toml` sits in the working directory. To see the thresholds a project will actually apply, pass the file: `stsmell rules -c stsmell.toml`.

03 Discover

`src/stsmell/discovery.py`

The smallest module in the tool, and the first one inside `analyze()`. `discover(root, excludes)` returns a sorted list of Tonel files. If `root` is itself a file it short-circuits, returning `[root]` when it qualifies and `[]` otherwise — that is what makes `stsmell analyze path/to/One.class.st` work with no special case upstream.

Otherwise it walks `root.rglob("*.st")` and keeps what passes two tests. `is_tonel_file()` checks the compound suffix against `(".class.st", ".trait.st", ".extension.st")` and explicitly rejects `package.st`, the Tonel package marker that carries no class or method. `_excluded()` then applies `fnmatch` against the union of `DEFAULT_EXCLUDES` — `.git`, `node_modules`, `.venv`, `pharo-local` — and any user patterns.

The result is sorted, which is not cosmetic: it makes findings deterministically ordered for a given tree, so baselines and CI diffs stay stable across runs and across machines.

04 Parse

`src/stsmell/parsing.py`

`parse_file()` reads the file as **bytes**, normalises it, and returns `(source, tree)`. Bytes rather than `str` throughout, because tree-sitter reports node positions as byte offsets; `node_text()` slices the same buffer and decodes with `errors="replace"` at the last moment.

The grammar and parser are built once and cached with `functools.lru_cache(maxsize=1)`. Loading a tree-sitter language is not free, and the pipeline parses thousands of files in one run.

Normalisation that preserves byte offsets

This is the most quietly clever part of the module. Traits mark class-side methods with `classSide >>` where classes use `class >>`, and the upstream grammar only accepts the latter – so trait files would fail to parse. `normalize_source()` rewrites the keyword before parsing.

The subtlety is the replacement: `classSide` (9 characters) becomes `class` plus *four spaces* (also 9 characters). Every byte offset after the rewrite is identical to the original file's, so every line and column tree-sitter reports still points at the right place in the source the user actually has on disk. A naive replacement would shift every subsequent offset and silently corrupt reported line numbers.

SRC/STSMELL/PARSING.PY

```
_CLASS_SIDE_RE = re.compile(
    rb"^(\\n)([A-Za-z_][A-Za-z0-9_]*)([ \\t]+)classSide([ \\t]*>>)"
)
_CLASS_SIDE_REPLACEMENT = rb"\\1\\2\\3class    \\4"    # 'class' + 4 spaces == 'classSide'
```

Errors are recorded, not fatal

`has_error(tree)` reports whether tree-sitter hit a syntax error. Crucially, `extract_file()` records this as a flag on `ParsedFile.parse_error` and carries on. tree-sitter recovers locally, so the rest of a partially-broken file still yields usable classes and methods. `build_project()` collects those paths and every report surfaces the count rather than hiding it – measured on the Pharo kernel at roughly 0.16% of files (14 of 8,864), caused by known upstream grammar gaps such as argument-only blocks and nested literal arrays.

05 Extract

src/stsmell/extract.py · src/stsmell/model.py

`extract_file()` walks the top-level children of the root node exactly once, maintaining one piece of state, and builds the dataclasses everything downstream consumes.

A Tonel file has a distinctive shape: an optional double-quoted class comment, then a STON map describing the class, then a run of method definitions. The extractor mirrors that shape directly.

<code>class_comment</code>	Held in a <code>pending_comment</code> local and attached to the <i>next</i> definition seen, then cleared. The comment precedes the definition in Tonel, so it cannot be read as a child of it.
<code>definition</code>	Dispatched through <code>_DEFINITION_KINDS</code> to <code>_build_class()</code> , which reads the STON header. An unwrapped definition node is tolerated as well.

method_definition

Built by `_build_method()` , then routed to its owning class by `_target_class()` .

Reading the STON header

Three small helpers do all the work. `_ston_pairs()` turns the map into `{key: value_node}` with the leading `#` stripped from each symbol key; `_ston_scalar()` unwraps a string, symbol or number; `_ston_list()` handles `#instVars : [ 'a' 'b' ]` , degrading gracefully to a single-element list when the value is a bare scalar.

One compatibility detail lives here. Tonel 3.0 replaced `#category` on class definitions with the pair `#package + #tag` , so `_build_class()` reads `package = scalar("package")` or `scalar("category")` and `category = scalar("category")` or `scalar("tag")` – both generations of the format land in the same fields.

Reconstructing selectors

A keyword method reference carries one `selector` field per keyword part. `_selector_of()` concatenates them, which is what turns the parsed fragments of `Account >> deposit: amount into: target` back into the selector `deposit:into: .` The parameters come from the parallel `param` fields, and `class_side` is simply whether the reference has a `class_side` child.

Orphan methods get a home

`_target_class()` matches a method to the class whose name it names. When nothing matches – a mismatched or renamed reference – and there is exactly one definition in the file, Tonel's one-type-per-file guarantee makes that the sensible target. When there is no definition at all, as in an `.extension.st` file, `extract_file()` synthesises a `ClassDef(kind="extension")` holder so the method is still measured and still judged, rather than silently disappearing.

WHY PARSEDFILE KEEPS THE TREE

`ParsedFile` stores the `tree` object even though nothing reads it directly. `tree-sitter Node` s are views into their `Tree` : if the tree is garbage-collected, every `MethodDef.body_node` derived from it becomes invalid. Holding the tree for the lifetime of the `Project` is what keeps stage 6 able to walk method bodies at all.

The model

Four dataclasses, deliberately plain. `MethodDef` carries the selector, params, temporaries, pragmas, category, path, line span, raw source and `body_node` , plus a `qualified_name` property that renders `Account class >> new` – the form Smalltalk developers actually read, and the string used as a finding's stable identity. `ClassDef` carries the STON header fields, the comment, a `kind` of `class / trait / extension` , and its methods, with `instance_methods / class_methods` views. `Project` is the whole run: `classes` , `methods` and `class_names()` flatten across every file, and that last one is what makes coupling measurable only against classes actually present in the analysed source.

§ Interlude: there is no control flow to count

src/stsmell/selectors.py

Before stage 6 makes sense, one property of the language has to be on the table, because it invalidates the standard way of computing half these metrics.

Smalltalk has no control-flow syntax. There is no `if`, no `while`, no `for`, no `switch`. Every branch and every loop is an ordinary message send that happens to take block arguments:

SMALLTALK – CONDITIONALS AND LOOPS ARE JUST MESSAGES

```
x > 0
  ifTrue: [ self grow ]
  ifFalse: [ self shrink ].

1 to: 10 do: [ :i | self step: i ].
```

Cyclomatic complexity in a C-family language is computed by counting keywords. Here there are no keywords to count – `ifTrue:ifFalse:` is a selector on `Boolean`, indistinguishable at the syntax level from `printOn:`. So complexity has to be computed by recognising a *curated set of selectors* on message-send nodes. That set is `selectors.py`, and it is the conceptual heart of the tool:

THE FOUR SETS THAT UNION INTO BRANCHING		
SET	SIZE	REPRESENTATIVE MEMBERS
CONDITIONAL	21	<code>ifTrue:</code> · <code>ifNil:ifNotNil:</code> · <code>caseOf:otherwise:</code> · <code>at:ifAbsent:</code> · <code>detect:ifNone:</code>
BOOLEAN	2	<code>and:</code> · <code>or:</code> – short-circuit, so each adds a path
LOOPING	23	<code>whileTrue:</code> · <code>to:do:</code> · <code>timesRepeat:</code> · <code>collect:</code> · <code>inject:into:</code>
EXCEPTIONAL	5	<code>on:do:</code> · <code>ensure:</code> · <code>ifCurtailed:</code> · <code>on:fork:</code>

The module carries three more sets used elsewhere in the pipeline: `ABSTRACT_MARKERS` (`subclassResponsibility`, `shouldBeImplemented`, `notYetImplemented`), which identify a method that refuses to implement itself; `SELF_RECEIVERS` (`self`, `super`, `thisContext`), which mark a receiver as *not* foreign; and the node-type sets that let metric code tell a message send from a cascaded one.

Everything in stage 6 that looks like a syntactic judgement is really a lookup against one of these sets.

06 Measure

src/stsmell/metrics/

`compute_metrics(project)` runs in a fixed order, because the later steps depend on the earlier ones. First the inheritance graph over the whole project; then, per class, one `MethodMetrics` per method; then the `ClassMetrics` that aggregates those method numbers together with the hierarchy and the set of known class names.

Hierarchy — what a depth of 1 means

`Hierarchy.build()` indexes every non-extension class by name, records children per superclass, notes superclasses that were never defined in `unresolved`, and computes a depth for each class.

The definition of depth is a deliberate choice worth stating plainly: `_compute_depth()` counts *links walked until the chain leaves the project*. A class whose superclass is `Object` — which lives in the Pharo image, not in the analysed source — has depth 1. The walk simply stops rather than guessing how deep the external hierarchy runs, and a `seen` set guards against a malformed cyclic hierarchy. The consequence: DIT is a measure of inheritance depth *you own*, which is the number a project can actually act on.

Method metrics — one walk, many counters

`compute_method_metrics()` makes a single `walk(body)` pass and accumulates everything from it. The interesting parts are the corrections, each of which exists because the naive version was wrong:

MLOC	<code>_code_lines()</code> collects line numbers from <i>leaf</i> nodes only, skipping <code>comment</code> nodes, and returns the size of the set. Counting <code>end_line - start_line</code> would include blank lines and comment blocks; counting all nodes would double-count.
CYCLO	Starts at 1 and adds one for every message whose selector is in <code>BRANCHING</code> . No syntax involved.
NBLK	<code>_block_depth()</code> recurses tracking maximum nesting of <code>block</code> nodes — which, given the interlude, is the Smalltalk equivalent of nested <code>if / for</code> bodies.
MAXCHAIN	<code>_chain_length()</code> counts sends stacked on one another by following the <code>receiver</code> field. <code>_chain_base()</code> finds the root of the chain, and <code>FOREIGN_MAXCHAIN</code> records the longest chain <i>not</i> rooted at self or super — the Law-of-Demeter number.
NOAV	Distinct lowercase identifiers accessed. Guarded by a precomputed <code>declared_ids</code> set so that the names inside a <code> a b </code> temporaries declaration count as declarations, not accesses, and do not inflate the number.

Two shape detectors run after the walk. `_detect_accessor()` recognises exactly the two canonical forms — a single `^ ivar` return with no parameters, or a single `ivar := arg` assignment with exactly one parameter whose name matches — and requires that the name really is an instance variable of this class. `_detect_abstract()` recognises a body that is a single send of an `ABSTRACT_MARKERS` selector to `self`, with or without a caret. Both feed cohesion and both feed rules.

A second short walk sets `calls_super_same_selector`, looking for a `super`-receiver send of the method's own selector — the classic override-and-extend shape.

Class metrics — and the cohesion measure

Most of `compute_class_metrics()` is aggregation: NOM and NOCM count methods, WMC sums method complexity, CLOC sums method lines, NIV counts instance variables, DIT and NOC come from the hierarchy. CBO is the one with a twist — the classes referenced across all methods, *intersected with the project's own class names* and minus the class itself, so referencing `OrderedCollection` from the image does not inflate coupling.

TCC, tight class cohesion, is the subtle one. It is the fraction of eligible method pairs that touch at least one instance variable in common:

SRC/STSMELL/METRICS/KLASS.PY

```
eligible = [
    m for m in metrics
    if not m.is_accessor and not m.is_abstract and not m.method.class_side
]
if len(eligible) < 2:
    return None
pairs = list(combinations(eligible, 2))
connected = sum(1 for a, b in pairs if a.accessed_ivars & b.accessed_ivars)
return connected / len(pairs)
```

Accessors and abstract methods are excluded because they trivially touch one variable or none and would otherwise dominate the score in either direction. Below two eligible methods the measure is undefined, and the code returns `None` rather than a misleading zero — which is precisely why the `god-class` rule opens with `if cm.tcc is None: return`. A class too small to have a cohesion score is not a god class.

07 Judge

src/stsmell/rules/

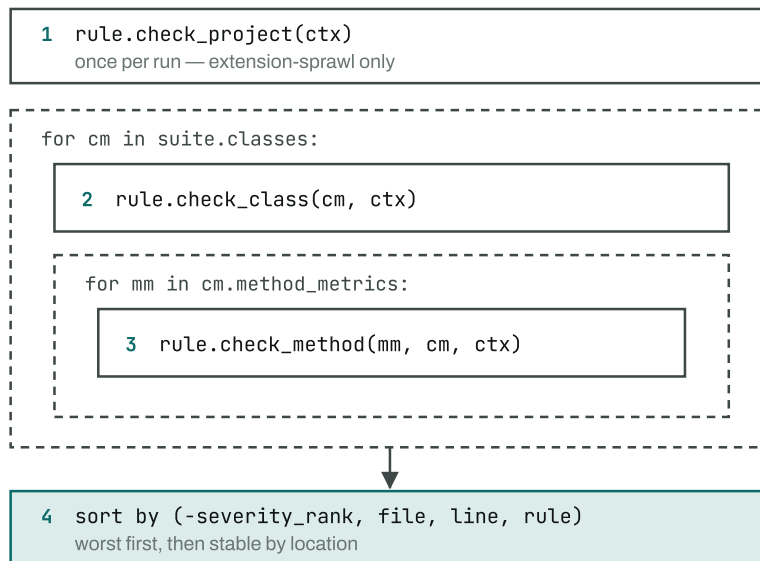
A rule is a subclass of `Rule` that declares an `id`, `name`, `severity`, `description` and a `defaults` dict, and overrides one of three hooks. The base class returns an empty tuple from all three, so a rule only implements the level it cares about.

<code>check_method(mm, cm, ctx)</code>	Once per method, with its class's metrics alongside.
--	--

<code>check_class(cm, ctx)</code>	Once per class.
-----------------------------------	-----------------

<code>check_project(ctx)</code>	Once per run, with the whole <code>MetricSuite</code> reachable through <code>ctx</code> . Only <code>extension-sprawl</code> uses it, because it aggregates across packages.
---------------------------------	---

Registration is by decorator: `@register` instantiates the class and appends it to a module-level list. `rules/__init__.py` imports the three rule modules purely for that side effect, so importing the package populates the registry. Threshold lookup goes through `self.opt(ctx, name)`, which reads the merged config and falls back to the rule's own default — a rule never reads `Config` directly.



Every active rule is offered every level.

The base class returns an empty tuple for the levels a rule ignores, so there is no registry of which rule is which.

Dispatch order. `run_rules()` filters to rules the config leaves enabled, then walks project → class → method. The final sort puts critical findings first and is fully determined by `(severity, file, line, rule id)`, so output is reproducible run to run.

The rule catalogue

Eighteen rules across three modules. The last three have no direct Java analogue — they exist because Smalltalk has failure modes that Java-derived linters have never needed to name.

RULES/METHOD_RULES.PY — EVALUATED PER METHOD			
RULE ID	SEVERITY	FIRES WHEN	DEFAULTS
long-method	MAJOR	MLOC exceeds the limit	max_lines = 25
complex-method	MAJOR	CYCLO exceeds the limit	max_cyclo = 8
deeply-nested-blocks	MAJOR	NBLK exceeds the limit	max_depth = 3
feature-envy	MAJOR	top foreign receiver ≥ minimum <i>and</i> > self sends	min_foreign_sends = 5
long-parameter-list	MINOR	PARAMS exceeds the limit	max_params = 4
too-many-temporaries	MINOR	TEMPS exceeds the limit	max_temps = 6
message-chain	MINOR	FOREIGN_MAXCHAIN exceeds the limit	max_chain = 4
uses-this-context	MINOR	the body references <code>thisContext</code>	—
cascade-abuse	INFO	MAX_CASCADE exceeds the limit	max_cascade = 6

RULES/CLASS_RULES.PY — EVALUATED PER CLASS			
RULE ID	SEVERITY	FIRES WHEN	DEFAULTS
god-class	CRITICAL	all three conditions hold at once	min_wmc = 47 · max_tcc = 0.33 · min_niv = 5
large-class	MAJOR	NOM exceeds the limit	max_methods = 40
refused-bequest	MAJOR	an inherited selector is overridden with <code>subclassResponsibility</code>	—
data-class	MINOR	all three conditions hold at once	min_accessor_ratio = 0.8 · min_methods = 5 · min_ivars = 2
deep-inheritance	MINOR	DIT exceeds the limit	max_depth = 8
high-coupling	MINOR	CBO exceeds the limit	max_cbo = 15

RULES/SMALLTALK_RULES.PY — NO JAVA ANALOGUE			
RULE ID	SEVERITY	FIRES WHEN	DEFAULTS
extension-sprawl	MAJOR	a package exceeds <i>either</i> limit, project-wide	max_extended_classes = 10 · max_extension_methods = 30
missing-class-comment	MINOR	a class or trait has no class comment	—
god-class-side	MINOR	NOCM exceeds the limit	max_class_methods = 15

The Smalltalk-native three are worth a sentence each. **extension-sprawl** is the only project-level rule: Pharo lets any package attach methods to any class, and at scale that makes behaviour impossible to locate, so the rule aggregates per package and flags monkey-patching by volume. **missing-class-comment** exists because Pharo treats the class comment as the primary documentation of a class's purpose and the community enforces its presence. **god-class-side** catches the Smalltalk shape of a static-utility dumping ground — behaviour piled onto the metaclass instead of instances.

08 Filter and report

```
cli.cmd_analyze() · src/stsmell/report/
```

Back in `cmd_analyze`, the findings pass two filters before rendering — in this order, which matters.

Baseline subtraction

`_load_baseline()` reads a JSON file written earlier by `stsmell baseline` and builds a set of `(rule, entity)` pairs. Any finding whose `key()` is in that set is dropped.

The key is deliberately *line-independent*. `Finding.key()` returns `(self.rule, self.entity)` – and `entity` is the qualified name, `Account >> deposit:`, not a file position. Editing a file above a known-smelly method shifts its line number; had the key included the line, every such edit would resurrect the whole baseline as new findings. This way a suppressed smell stays suppressed until the method is renamed or fixed.

Severity floor

Then `SEVERITY_ORDER[config.min_severity]` gives an integer floor and anything below it is dropped. The order is `info` 0, `minor` 1, `major` 2, `critical` 3, and the default is `info` – so by default nothing is filtered. Because this runs *after* baseline subtraction, the exit code reflects exactly what was printed.

Three renderers

<code>render_text</code>	Groups by file, prints line, severity, message and rule id, with the entity on a dimmed second line, then a summary counting findings by severity plus files, classes and methods scanned. Colour is ANSI, enabled only when the stream is a TTY and <code>NO_COLOR</code> is unset. Paths are printed relative to the analysed root.
<code>render_json</code>	A <code>summary</code> object (files, classes, methods, findings, parse errors, read errors), the list of parse-error paths, and every finding with its <code>metrics</code> dict – the numbers that triggered it, so a consumer can re-check the judgement.
<code>render_sarif</code>	SARIF 2.1.0 for GitHub code scanning. Emits rule metadata for the rules that actually fired, and locations as root-relative POSIX URIs.

Two details in the SARIF writer are easy to miss. SARIF has three levels where `stsmell` has four severities, so `_LEVEL` maps both `critical` and `major` onto `error`, `minor` onto `warning` and `info` onto `note`. And the message joins entity and text with an em dash rather than a colon – because keyword selectors already end in one, and `classify:: complexity is 11` reads as a typo.

The other three commands

`cmd_metrics` runs the identical pipeline but ignores findings, emitting `as_dict()` rows for either scope. CSV field names are taken from the first row's keys via `csv.DictWriter`, so adding a metric to `as_dict()` automatically adds a column. This is the command you use to recalibrate thresholds against your own codebase rather than inheriting the Pharo-kernel defaults.

`cmd_rules` prints or serialises the registry. `cmd_baseline` writes `{version, findings:[{rule, entity, message}]}` – note it stores no line numbers, matching the key design above – to a file, or to stdout when `-o` is omitted, with the confirmation line going to stderr so redirecting stdout yields clean JSON.

§ A finding, end to end

tests/fixtures/smelly/ComplexMethodExample.class.st

All output below is real, produced by the version of the tool in this repository. The input is one fixture file:

THE SOURCE

```
"I contain one method with far too many branches."
Class {
    #name : 'ComplexMethodExample',
    #superclass : 'Object',
    #instVars : [ 'value' ],
    #package : 'Smelly-Core'
}

{ #category : 'evaluating' }
ComplexMethodExample >> classify: input [
    input isNil ifTrue: [ ^ #none ].
    input isString ifTrue: [ ^ #string ].
    input isNumber ifTrue: [ ^ #number ].
    input isSymbol ifTrue: [ ^ #symbol ].
    (input isArray and: [ input isEmpty ]) ifTrue: [ ^ #emptyArray ].
    (input isArray or: [ input isDictionary ]) ifTrue: [ ^ #collection ].
    input isCharacter ifTrue: [ ^ #character ].
    input isBoolean ifTrue: [ ^ #boolean ].
    ^ #unknown
]
```

3 DISCOVER

The path ends in `.class.st`, so `is_tonel_file()` accepts it. Given directly as `root`, `discover()` short-circuits and returns a one-element list.

4 PARSE

No `classSide >>` appears, so `normalize_source()` returns the bytes unchanged. `has_error()` is false – the file parses cleanly.

5 EXTRACT

The leading string becomes `pending_comment` and attaches to the class. The STON map yields `name`, `superclass`, `instVars = ['value']` and `package`. `_selector_of()` reconstructs the selector `classify:` with one parameter `input`; `_target_class()` matches it to the class by name.

6 MEASURE

One walk of the body produces the numbers below. CYCLO is `1 + 8 ifTrue: + 1 and: + 1 or: = 11`; the ten unary predicates (`isNil`, `isString`, ...) are not in `BRANCHING` and contribute nothing. All ten of those unary sends go to `input`, a parameter and therefore an envy candidate, while nothing is sent to `self`.

```
$ STSMELL METRICS ...COMPLEXMETHODEXAMPLE.CLASS.ST --FORMAT JSON
```

```
"MLOC": 9,          "CYCLO": 11,          "MSG": 20,
"PARAMS": 1,        "TEMPS": 0,          "NBLK": 1,
"MAXCHAIN": 2,      "FOREIGN_MAXCHAIN": 2,
"NOAV": 1,          "MAX_CASCADE": 0,  "SELF_SENDS": 0,
"TOP_FOREIGN": "input", "TOP_FOREIGN_SENDS": 10,
"IS_ACCESSOR": false, "IS_ABSTRACT": false
```

7 JUDGE

Two rules fire. `complex-method`: `CYCLO 11 > 8`. `feature-envy`: 10 foreign sends ≥ 5 and $10 > 0$ self sends. Everything else stays quiet – `MLOC 9` is under 25, `PARAMS 1` is under 4, `NBLK 1` is under 3, and `god-class` returns immediately because `TCC` is `None` for a single-method class.

8 REPORT

Both findings are `major`, so they sort together and pass the default `info` floor. The process exits 1.

```
$ STSMELL ANALYZE TESTS/FIXTURES/
```

```
smelly/ComplexMethodExample.class.st
9    major    cyclomatic complexity is 11 (limit 8) [complex-method]
      ComplexMethodExample >> classify:
9    major    sends 10 messages to 'input' but only 0 to self [feature-envy]
      ComplexMethodExample >> classify:
```

READ THE SECOND FINDING CRITICALLY

The `feature-envy` hit here is defensible but arguable: `classify:` is a type dispatch, not delegation, and the "move this method onto `input`" advice implied by the smell is not actionable when `input` is `Object`. This is the honest shape of threshold-based analysis – the metric is correct, the inference is a heuristic. It is also a good illustration of why `stsmell baseline` exists.

Where the numbers came from

calibration, and two rules that were deliberately weakened

The thresholds are not imported from a Java linter. They were derived by running the metric layer over the Pharo kernel – 8,864 classes, 91,273 methods – and reading the distribution:

PHARO KERNEL DISTRIBUTION • DEFAULTS SIT AROUND P95-P99						
METRIC	MEDIAN	P90	P95	P99	MAX	DEFAULT THRESHOLD
MLOC	2	9	13	26	1961	> 25
CYCLO	1	3	4	7	38	> 8
PARAMS	0	2	2	4	15	> 4
NBLK	0	1	2	3	8	> 3
NOM	4	24	39	91	812	> 40
WMC	6	35	62	150	1520	≥ 47
DIT	5	7	8	9	12	> 8

Two rows carry most of the argument. **The median Pharo method is two lines long** – a Java-style 50-line threshold would never fire, which is why `max_lines` is 25 and still stricter than it looks. And **the median class already sits at DIT 5**, deep in a hierarchy that a Java codebase would consider pathological, which is why `deep-inheritance` has to tolerate 8.

Percentile calibration defines *normal*, not *good*: the Pharo kernel contains genuinely old and genuinely smelly code, and the defaults inherit that. `stsmell metrics` exists so a project can redo this exercise against its own source.

Two ports that were wrong, and what replaced them

The same re-derivation applies to detection logic, not just numbers. Two rules were deliberately narrowed after being tested against the kernel.

Refused Bequest. The standard Java heuristic is "a subclass never touches its inherited fields". In Smalltalk that misfires catastrophically, because idiomatic code reaches inherited state through accessors – `self origin`, not `origin` – so the variable name never appears. On the Pharo corpus the heuristic produced **3,707 findings against 16 real ones**. The implemented rule keeps only the precise, unambiguous form: a subclass that overrides an inherited selector with `subclassResponsibility`. It also returns early when the superclass is not in the analysed source, because it genuinely cannot tell. The infrastructure for the rejected heuristic is still there – `Hierarchy.inherited_ivars()` – but nothing calls it.

Feature Envy. Counting every send to a non-`self` receiver blames the method for objects it built itself: `parts := OrderedCollection new. parts add: ...` looks like five sends to a foreign receiver. Two restrictions fix it. Only instance variables and parameters are envy candidates, so locals the method constructed are excluded; and binary sends are skipped entirely, because `amount > 0` is arithmetic, not delegation. A send to a capitalised global is also excluded – that is construction.

```

receiver = _receiver(node)
if receiver is not None:
    if receiver.type in SELF_RECEIVERS:
        m.self_sends += 1
    elif receiver.type == "identifier" and kind != "binary_message":
        name = node_text(receiver, source).strip()
        if name in ivars or name in envy_names:      # envy_names = params
            m.foreign_sends[name] += 1

```

§ Extending and testing

Adding a rule

Four steps, no central registration to edit. Subclass `Rule` in whichever of the three rule modules fits; declare `id`, `name`, `severity`, `description` and `defaults`; decorate with `@register`; override the one hook you need and yield findings through `self.method_finding()` or `self.class_finding()`, which fill in entity, file and line from the model.

THE SHAPE OF A RULE

```

@register
class TooManyTemporaries(Rule):
    id = "too-many-temporaries"
    name = "Too Many Temporaries"
    severity = "minor"
    description = "Many temporary variables suggest the method does several jobs."
    defaults = {"max_temps": 6}

    def check_method(self, mm, cm, ctx) → Iterable[Finding]:
        limit = self.opt(ctx, "max_temps")
        if mm.temps > limit:
            yield self.method_finding(
                mm, f"declares {mm.temps} temporaries (limit {limit})",
                TEMPS=mm.temps, limit=limit,
            )

```

The keyword arguments become the finding's `metrics` dict, which surfaces in JSON output. Include the observed value and the limit – that is what lets a reader check the verdict without rerunning the tool. If the rule needs a metric that does not exist yet, add it to `MethodMetrics` or `ClassMetrics` and to the matching `as_dict()`, and it appears as a new `stsmell metrics` column for free.

The test suite

57 tests, layered the same way the code is, so a failure localises immediately.

tests/test_extract.py

CST → model. Selector reconstruction, class-side detection, STON fields, comments, orphan methods.

<code>tests/test_metrics.py</code>	Model → numbers. Complexity counting, accessor and abstract detection, cohesion, hierarchy depth.
<code>tests/test_rules.py</code>	Numbers → findings. Constructs metrics directly, no parsing – this is the layering law paying off.
<code>tests/test_cli.py</code>	End to end. Formats, exit codes, config resolution, baseline suppression.
<code>tests/fixtures/</code>	Real Tonel files, one per smell, plus <code>clean/</code> to catch false positives – the fixture set is the regression suite for precision.

```
$ .VENV/BIN/PYTHON -M PYTEST TESTS/ -Q
```

```
..... [100%]
57 passed in 0.19s
```

Known limitations

- **Tonel only.** The older `! -chunk` fileOut format is not supported.
- **Roughly 0.16% of Pharo kernel files fail to fully parse** (14 of 8,864) due to upstream grammar gaps: argument-only blocks such as `[ :a ]`, `##literal` bindings, non-ASCII symbols, nested literal arrays. tree-sitter recovers, the rest of each file is still analysed, and the count is always reported rather than hidden.
- **No image, so no send resolution.** Coupling is name-based and confined to classes present in the analysed source; a selector cannot be resolved to an implementation the way it could inside a live Pharo image.
- **Trait `classSide >>` is rewritten before parsing.** Offsets are preserved exactly, so reported positions stay correct.

```
stsmell 0.1.0 · architecture and control flow
```

```
Every code excerpt, metric value and terminal transcript in this document was taken from the source and from live runs of the tool in this repository.
```